

# PROGRAMACIÓN I

Presencial **Turno noche**

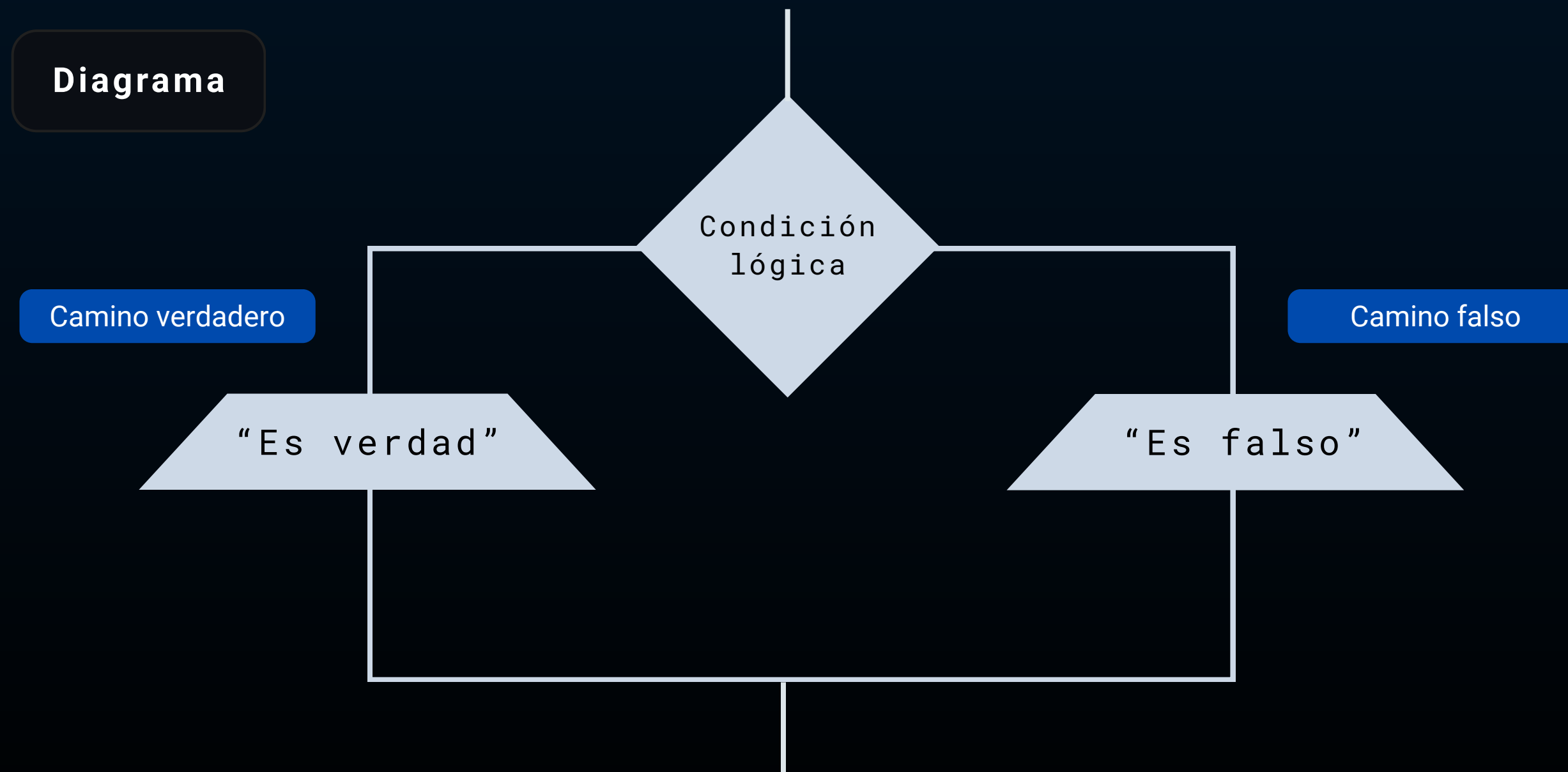
Codificación

# Estructura de control

02. Condicionales - Decisión simple

# Decisión simple

La **decisión simple** permite que el programa *elija* entre dos posibles caminos de ejecución. La ejecución se determina luego de evaluar la **proposición lógica** que contiene la decisión simple: si es **verdadera**, el flujo de ejecución toma un camino, y si es **falsa**, otro.



# Operadores relacionales

Para poder realizar proposiciones lógicas es necesario hacer uso de operadores relacionales.

| Operador | Significado       |
|----------|-------------------|
| >        | Mayor que         |
| <        | Menor que         |
| >=       | Mayor o igual que |
| <=       | Menor o igual que |
| ==       | Igual que         |
| !=       | Distinto que      |

```
A > B
B < 12
A >= 0
A != B
B == 20
```



```
A > B > C
B = A
```



```
10 > 12
true
```



# Operadores lógicos

Para poder evaluar más de una **proposición lógica** podemos hacer uso de **operadores lógicos**.

| Operador | Significado |
|----------|-------------|
| &&       | AND - Y     |
|          | OR - O      |
| !        | NOT - NO    |

```
A > B && A > C
A < 0 || B < 0
!(A > 0)
```



```
(A > B) && > C
```



```
A < 0 && A > 0
```





# Tabla de **verdad**

Indican el **valor lógico** de una **proposición** para cada combinación posible de sus variables

OPERADOR && (AND)

| A     | B     | A && B |
|-------|-------|--------|
| true  | true  | true   |
| false | true  | false  |
| true  | false | false  |
| false | false | false  |

# Tabla de **verdad**

Indican el **valor lógico** de una **proposición** para cada combinación posible de sus variables

OPERADOR **||** (OR)

| A     | B     | A <b>  </b> B |
|-------|-------|---------------|
| true  | true  | true          |
| false | true  | true          |
| true  | false | true          |
| false | false | false         |

# Tabla de **verdad**

Indican el **valor lógico** de una **proposición** para cada combinación posible de sus variables

OPERADOR ! (NOT)

| A     | !A    |
|-------|-------|
| true  | false |
| false | true  |



Ejercicio

# Ejercicio en diagrama y código

# Ejercicio

Hacer un programa para ingresar por teclado **un número** y emitir por pantalla un mensaje aclaratorio que **indique si el mismo es positivo**, caso contrario no emitir nada.



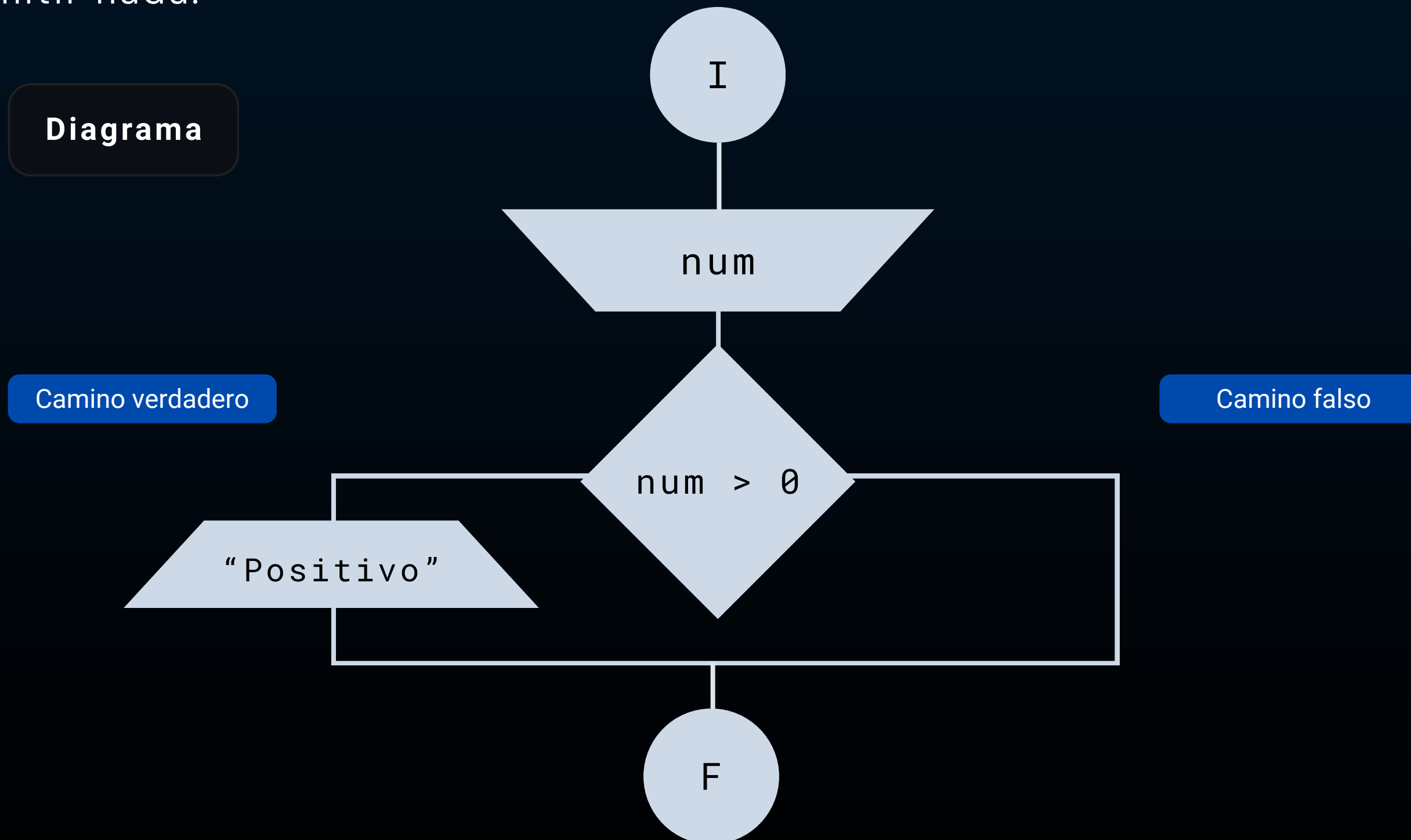
# Ejercicio

Hacer un programa para ingresar por teclado **un número** y emitir por pantalla un mensaje aclaratorio que **indique si el mismo es positivo**, caso contrario no emitir nada.



# Ejercicio

Hacer un programa para ingresar por teclado **un número** y emitir por pantalla un mensaje aclaratorio que **indique si el mismo es positivo**, caso contrario no emitir nada.



# Ejercicio

Hacer un programa para ingresar por teclado **un número** y emitir por pantalla un mensaje aclaratorio que **indique si el mismo es positivo**, caso contrario no emitir nada.

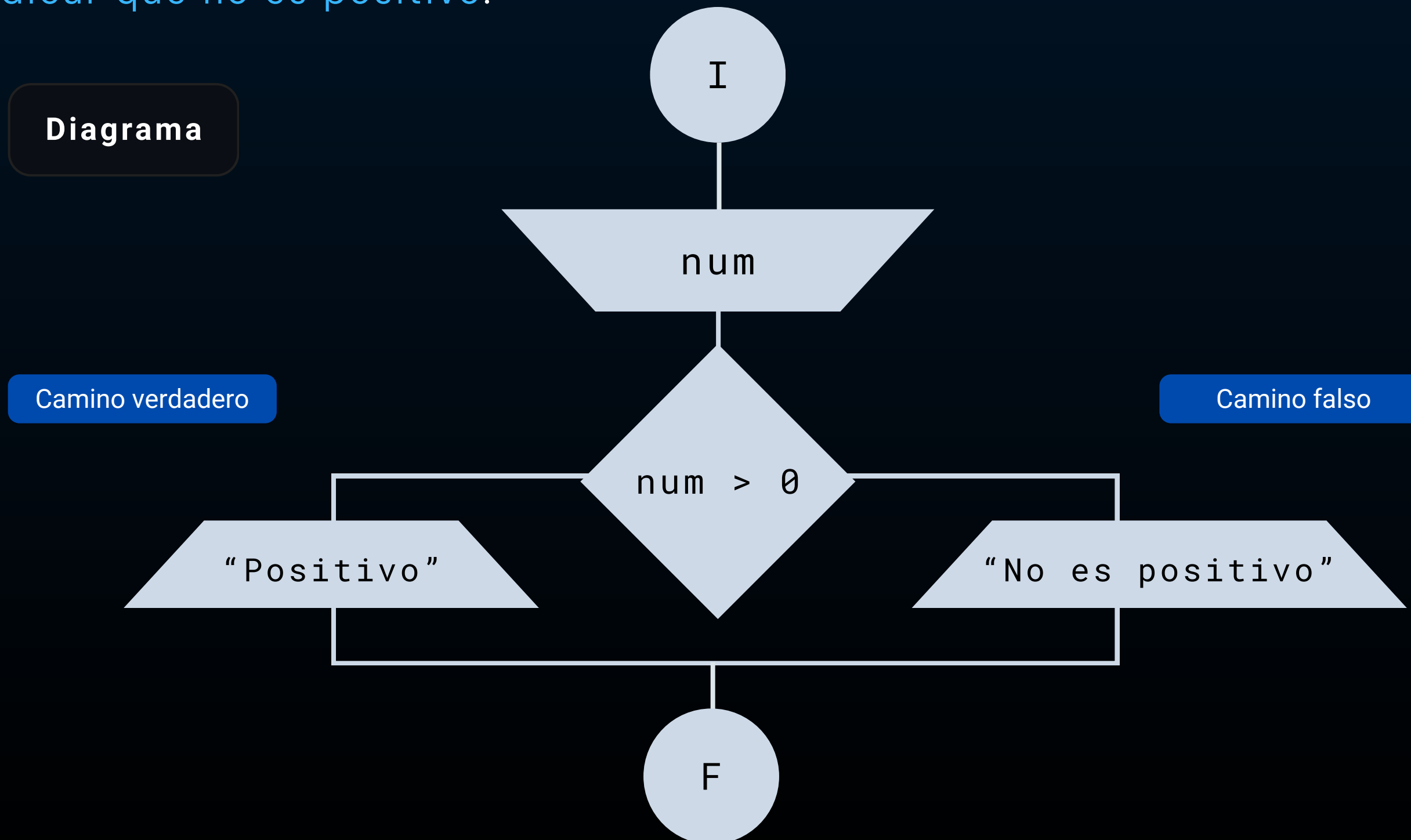
## Código

```
int num;  
cin >> num;  
  
if(num > 0)  
{  
    cout << "Positivo";  
}
```



# Ejercicio

Hacer un programa para ingresar por teclado **un número** y emitir por pantalla un mensaje aclaratorio que **indique si el mismo es positivo**, caso contrario **indicar que no es positivo**.

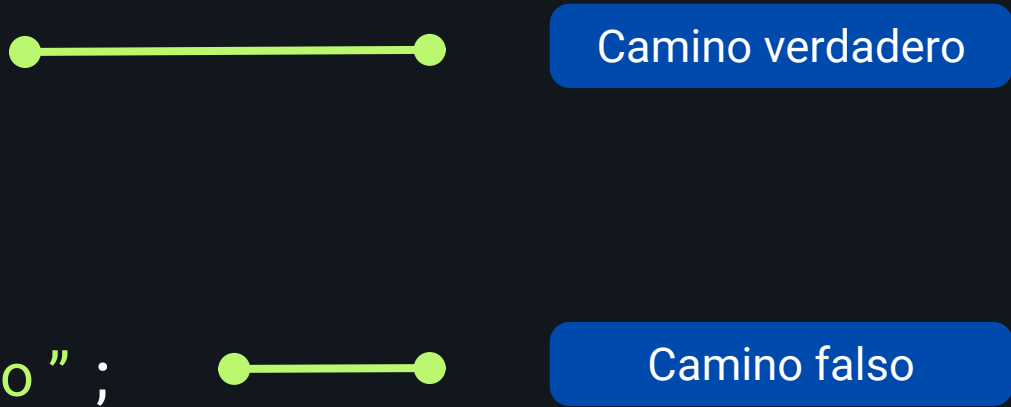


# Ejercicio

Hacer un programa para ingresar por teclado **un número** y emitir por pantalla un mensaje aclaratorio que **indique si el mismo es positivo**, caso contrario **indicar que no es positivo**.

## Código

```
int num;  
cin >> num;  
  
if(num > 0)  
{  
    cout << "Positivo";  
}  
else  
{  
    cout << "No es positivo";  
}
```



Codificación

# Estructura de control

02. Condicionales - Decisión múltiple



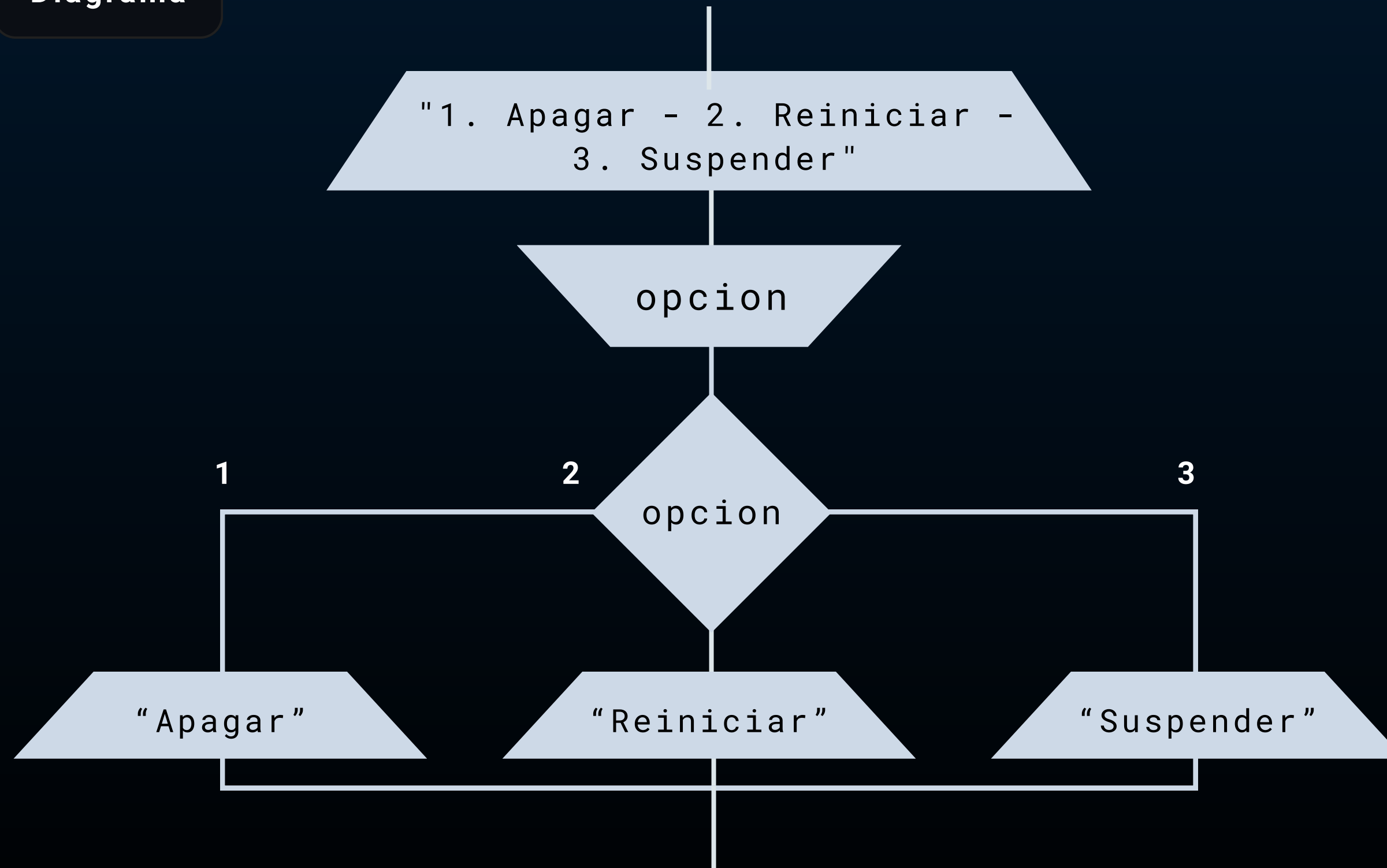
# Decisión múltiple

La sentencia `switch` evalúa una `expresión` y ejecuta un bloque de código según el valor que tome, eligiendo entre `múltiples opciones` definidas con `case`.

Supongamos que queremos simular el menú de apagado de una computadora, donde el usuario elige entre `Apagar`, `Reiniciar` o `Suspender`. La sentencia `switch` nos permite ejecutar una acción diferente según la opción elegida.

# Deicisión múltiple

## Diagrama



# Deicisión múltiple

## Código

```
int opcion;

cout << "1. Apagar - 2. Reiniciar - 3. Suspende";
cin >> opcion;

switch (opcion) {
    case 1:
        cout << "Apagando el sistema...";
        break;
    case 2:
        cout << "Reiniciando...";
        break;
    case 3:
        cout << "Suspensión en curso...";
        break;
    default:
        cout << "Opción no válida.";
}
```